

方案 B 变体（对等 Mesh）实施变更细节建议方案

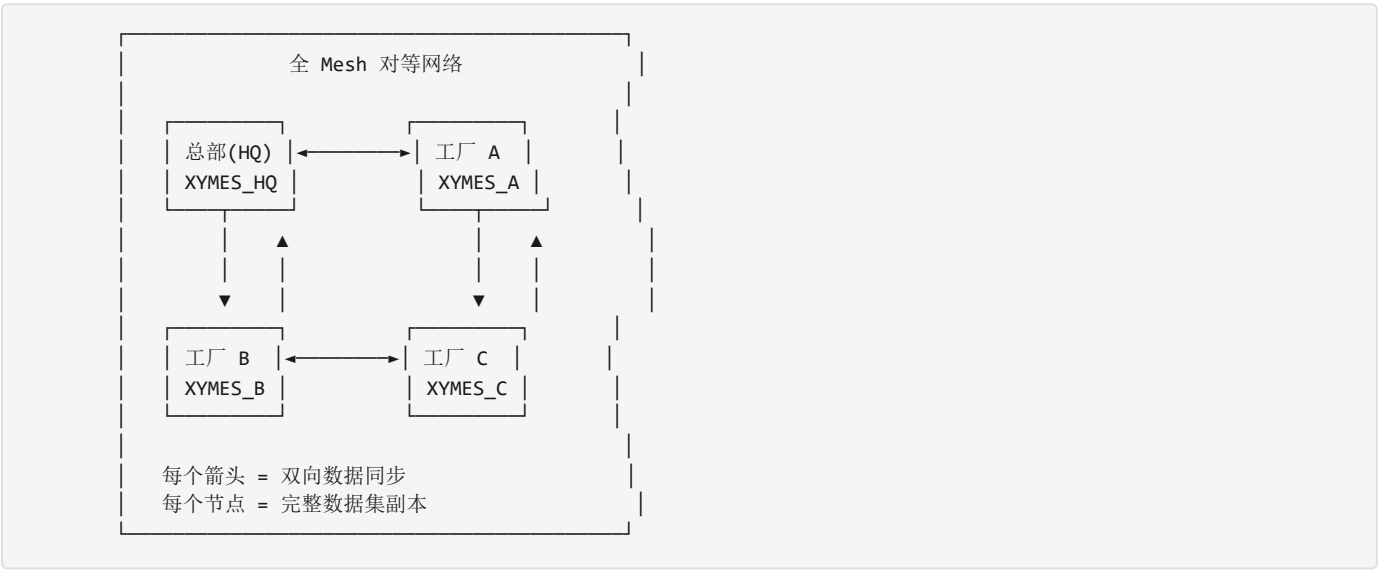
Seals MES System — 异地分布式部署实施规划 | 2026年5月

一、方案概述

1.1 核心理念

总部不设独立中心数据库，总部和所有工厂地位**完全对等**。每个节点（含总部）都拥有完整的本地 SQL Server 实例，节点之间**相互同步**全部业务数据。总部只是网络中的一个“普通节点”，通过自身本地库即可查看全系统汇总报表。

1.2 拓扑结构



1.3 与原方案 B 的关键区别

维度	原方案 B (星型)	方案 B 变体 (对等 Mesh)
中心节点	总部有独立中心库	无中心节点，全员对等
数据流向	工厂→总部单向上报	所有节点互相同步
总部角色	数据汇总者+主数据下发者	普通节点，与其他厂相同
总部断网	无法汇总，但各厂正常	总部离线后恢复时补同步
主数据维护	总部独占修改	任一节点可改，变更广播
每个节点数据量	总部全量，工厂仅自己	每个节点全量
跨厂查询	需汇总库/Linked Server	查本地库，零延迟

二、系统架构层变更

2.1 新增模块与文件

新增路径	职责	优先级
server/src/config/site.config.ts	站点标识配置：读取 SITE_PREFIX , SITE_NAME , DEPLOY_MODE , SYNC_PEERS	P0
server/src/config/sync.config.ts	同步策略配置：同步间隔、重试策略、批量大小、同步表清单	P1
server/src/modules/sync/*	数据同步模块：push / pull / snapshot / peer discovery 四个控制器 + service	P1
server/src/modules/conflict/*	冲突处理模块：冲突检测、LWW 自动合并、人工仲裁接口	P1
server/src/modules/report-aggregation/*	汇总报表模块：按 site_prefix 分组聚合查询，替代跨库查询	P1
server/src/middleware/siteGuard.middleware.ts	站点写入守卫：拦截非归属站点对业务数据的写操作	P0

2.2 数据库配置层改造

当前状态

文件：server/src/config/database.ts （31行），单库直连，连接池 max:10, min:2

变更内容

改造项	现状	变更
连接配置	硬编码 host: process.env.DB_HOST '127.0.0.1'	不变（每个节点连自己的本地库）
连接池	max: 10, min: 2	适当上调 max: 15 （同步服务额外占用连接）
超时配置	requestTimeout: 120000	增加 connectTimeout: 30000，同步推送到远端 peer 时超时保护

2.3 应用入口层改造

app.ts 变更

改造项	位置	变更
健康检查扩展	L73-93	返回 site_prefix , sync_status , last_sync_at
同步路由注册	L95 前	新增 routes.use('/sync', syncRoutes)
站点守卫中间件	全局中间件	在 authenticate 之后注册 siteGuard，为所有写操作注入 site_prefix 校验
同步服务启动	L98-106 旁	新增 startSyncService() 启动同步轮询/监听

server.ts 变更

改造项	变更
实例标识	启动时打印 SITE_PREFIX 和 SYNC_PEERS 配置
注册到 Mesh	启动时向 SYNC_PEERS 中各节点发送 POST /api/sync/announce 声明本节点上线

三、编号生成层改造（核心改造，影响面最广）

3.1 当前问题

所有编号生成函数都使用 `SELECT MAX(xxx_number) as max_num FROM 表 WHERE xxx_number LIKE :prefix` 模式。在单库场景下不会冲突，但在对等 Mesh 中：

- 每个节点只写入自己 `site_prefix` 的数据，编号查询时需增加 `WHERE site_prefix = :sitePrefix` 条件
- 编号格式需加入站点前缀，如 `M-HF-20260515-001`

3.2 改造方式

改造前（以 `generateProductionNumber` 为例）：

```
prefix = 'M' + YYYYMMDD
SELECT MAX(production_number) FROM Production_plan WHERE production_number LIKE prefix+'%'
→ M20260515001
```

改造后：

```
sitePrefix = process.env.SITE_PREFIX    // 如 'HF'
prefix = 'M-' + sitePrefix + '-' + YYYYMMDD + '-'
SELECT MAX(production_number) FROM Production_plan
  WHERE production_number LIKE prefix+'%' AND site_prefix = :sitePrefix
→ M-HF-20260515-001
```

建议：先抽取一个统一的编号生成基础函数 `generateNumber(table, column, prefix, sitePrefix, transaction)`，减少重复代码。统一改造入口后再加站点前缀，避免遗漏。

3.3 documentNumber.service.ts 中的 18 个生成函数

#	函数名	当前前缀格式	改造后格式
1	generateProductionNumber	M20260515	M-HF-20260515
2	generateOrderNumber	P20260515	P-HF-20260515
3	generateWRNumber	WR-20260515	WR-HF-20260515
4	generateTaskNumber	PT-20260515	PT-HF-20260515
5	generatePrepNumber	MP-20260515	MP-HF-20260515
6	generateInboundOrderNumber	PI-20260515	PI-HF-20260515
7	generateShippingOrderNumber	SM-20260515	SM-HF-20260515
8	generateForecastNumber	SF-20260515	SF-HF-20260515
9	generateLinesideTxnNumber	LS-20260515	LS-HF-20260515
10	generateOutsourcingReqNumber	OSR-20260515	OSR-HF-20260515
11	generateOutsourcingOrderNumber	OS-20260515	OS-HF-20260515
12	generateOutsourcingIssueNumber	OMI-20260515	OMI-HF-20260515
13	generateOutsourcingReceiptNumber	ORC-20260515	ORC-HF-20260515
14	generateOutsourcingInspectionNumber	OQI-20260515	OQI-HF-20260515
15	generateOutsourcingSettlementNumber	OST-20260515	OST-HF-20260515
16	generateOutsourcingReturnStockinNumber	ORS-20260515	ORS-HF-20260515
17	generateBackflushTaskNumber	BT-20260515	BT-HF-20260515
18	generateSemiProductionInboundOrderNumber	SPIO-20260515	SPIO-HF-20260515

3.4 Controller 中内联的编号生成 (27处, 未走 documentNumber.service.ts)

这些需要**先迁移到 documentNumber.service.ts**, 再统一改造:

#	文件路径	内联编号字段
1	purchasing/purchaseReq/purchaseReq.controller.ts	purchase_req_number , purchase_order_number
2	purchasing/purchaseOrder/purchaseOrder.controller.ts	purchase_order_number
3	purchasing/receivingNotice/receivingNotice.controller.ts	receiving_number , stock_in_number
4	purchasing/purchaseReturn/purchaseReturn.controller.ts	return_number
5	purchasing/purchasePrice/purchasePrice.controller.ts	price_list_number
6	purchasing/purchaseCalc/purchaseCalc.controller.ts	purchase_req_number
7	purchasing/purchaseInvoice/purchaseInvoice.controller.ts	invoice_number
8	sales/salesOrder/salesOrder.controller.ts	sales_order_number
9	sales/returnOrder/returnOrder.controller.ts	return_order_number
10	sales/salesPrice/salesPrice.controller.ts	price_list_number
11	sales/salesInvoice/salesInvoice.controller.ts	invoice_number
12	sales/shippingRequest/shippingRequest.controller.ts	request_number
13	production/materialPreparation/materialPreparation.controller.ts	preparation_number
14	production/materialReturn/materialReturn.controller.ts	return_number
15	production/outsourcingPrice/outsourcingPrice.controller.ts	price_list_number
16	production/pieceRateWage/pieceRateWage.controller.ts	wage_number
17	warehouse/stockIn/stockIn.controller.ts	stock_in_number
18	warehouse/stockCount/stockCount.controller.ts	count_number
19	warehouse/packingOrder/packingOrder.controller.ts	packing_number
20	warehouse/scrapDisposal/scrapDisposal.controller.ts	disposal_number
21	warehouse/abnormalIO/abnormalIO.controller.ts	request_number
22	quality/productionInspection/productionInspection.controller.ts	inspection_number
23	quality/nonconformingProduct/nonconformingProduct.controller.ts	nonconforming_number , stock_in_number
24	quality/reworkOrder/reworkOrder.controller.ts	rework_order_number
25	quality/scrapInboundOrder/scrapInboundOrder.controller.ts	stock_in_number
26	finance/standardCost/standardCost.controller.ts	cost_list_number
27	finance/pieceRatePrice/pieceRatePrice.controller.ts	price_list_number

工作量评估：18 个 service 函数改造 + 27 处内联迁移 = **约 45 个改造点**，是全部工作量中占比最大的一项（约 50%）。

四、数据库表结构变更

4.1 所有业务主表新增字段

新增字段	类型	默认值	说明
site_prefix	NVARCHAR(10) NOT NULL	当前站点前缀	数据归属标识，只有归属站点可写入
version	INT NOT NULL	1	乐观锁版本号，同步时冲突检测用
last_synced_at	DATETIME NULL	NULL	最后一次同步时间戳

SQL 示例：

```
-- 所有业务表执行以下迁移
ALTER TABLE production_order ADD site_prefix NVARCHAR(10) NOT NULL DEFAULT 'HQ';
ALTER TABLE production_order ADD version INT NOT NULL DEFAULT 1;
ALTER TABLE production_order ADD last_synced_at DATETIME NULL;

-- 为 site_prefix 创建索引（加速按站点过滤和同步查询）
CREATE INDEX IX_production_order_site ON production_order(site_prefix);
```

4.2 新增系统表

表名	用途	核心字段
sync_log	变更日志	id, table_name, record_id, operation(C/U/D), payload(JSON), source_site, target_site, status(pending/synced/failed), retry_count, created_at
sync_conflict_log	冲突记录	id, table_name, record_id, local_version, remote_version, local_data, remote_data, resolution(pending/auto_resolved/manual), created_at
sync_peer_status	各 peer 连接状态	peer_url, status(online/offline), last_sync_at, last_error, created_at, updated_at
sync_checkpoint	同步进度检查点	peer_url, table_name, last_synced_id, last_synced_at

4.3 涉及的业务表数量

根据 IDENTITY(1,1) 和 CREATE TABLE 搜索结果，约 **40+ 张业务表** 需要加 site_prefix + version 字段，主要涉及：

- 计划域：Production_plan, production_order, process_task, material_preparation, backflush_task
- 生产域：work_report, production_inbound_order, semi_production_inbound_order, production_inspection, production_inspection_defect
- 采购域：purchase_req, purchase_order, purchase_receiving_notice, purchase_return, purchase_price_list, purchase_invoice, purchase_quality_inspection, purchase_inspection_defect
- 销售域：sales_order, sales_forecast, shipping_request, shipping_order, return_order, sales_price_list, sales_invoice
- 仓库域：stock_in, stock_count, packing_order, scrap_disposal, abnormal_io_request, material_return, inventory_transaction, material_inventory_transaction, lineside_inventory_transaction, rework_order, nonconforming_product
- 委外域：outsourcing_req, outsourcing_order, outsourcing_material_issue, outsourcing_receipt, outsourcing_inspection, outsourcing_settlement, outsourcing_return_stockin, outsourcing_price_list
- 财务域：standard_cost_header, piece_rate_price_header, piece_rate_wage_header
- 基础数据域：item_master, customer, supplier, bom_header, routing_header, warehouse, work_center 等（主数据同样需归属标识）

五、事件系统层改造

5.1 EventBus 改造

当前文件：`server/src/shared/eventBus.ts`（66行），基于 Node.js EventEmitter 的进程内事件总线

改造项	变更
事件发布	每次业务事件 <code>emitAsync()</code> 后，同时写入 <code>sync_log</code> 表（变更捕获）
新增事件	<code>SYNC_CONFLICT_DETECTED</code> ， <code>SYNC_COMPLETED</code> ， <code>SYNC_PEER_OFFLINE</code>

5.2 EventStore 改造

当前文件：`server/src/services/eventStore.service.ts`（139行），`system_events` 表

改造项	变更
事件记录	增加 <code>source_site</code> 字段
事件消费	增加分布式锁（避免多节点重复消费同一事件）

5.3 EventWorker 改造

当前文件：`server/src/services/eventWorker.service.ts`（149行），轮询消费 pending 事件

改造项	变更
分布式锁	执行前获取数据库级锁（ <code>sp_getapplock</code> ），仅一个节点执行
<code>site_prefix</code> 过滤	避免重复消费非本站事件

5.4 定时任务防重

当前文件：`server/src/services/stockCountScheduler.service.ts`（node-cron 单进程执行）

改造项	变更
分布式锁	执行前获取数据库级锁（ <code>sp_getapplock</code> ），仅一个节点执行
同步结果	盘点结果通过 <code>sync_log</code> 同步到其他节点，其他节点不重复执行

六、文件上传层改造

当前文件：`server/src/middleware/upload.middleware.ts`（66行），本地 `uploads/` 目录

改造项	变更
文件元数据	新增 <code>file_attachment</code> 表，记录文件名、站点、同步状态
同步策略	方案1（轻量）： <code>rsync</code> 定时同步；方案2：MinIO/S3 对象存储；方案3：API 拉取

建议：初期采用方案1，`rsync` 每5分钟同步 `uploads/` 目录，实施成本最低。

七、前端变更

7.1 API 地址动态化

当前文件：`client/src/utils/request.ts` , `baseUrl: '/api/v1'`

改造项	变更
API 地址	改为 <code>import.meta.env.VITE_API_BASE '/api/v1'</code>
站点标识请求头	请求头增加 <code>X-Site-Prefix</code> , 后端可据此过滤数据

7.2 全局站点标识展示

改造项	变更
登录后	顶部导航栏显示当前站点名称 (如"合肥工厂")
页面标题	列表页可选按 <code>site_prefix</code> 过滤显示 (仅本厂 / 全部站点)
只读提示	非 <code>site_prefix</code> 归属的数据行, 操作按钮灰显 + 提示"此数据归属XX站点, 仅可查看"

7.3 移动端

`client-mobile/src/api/request.ts` 同理改造 API 地址动态化和站点展示。

八、日常数据变更细节

8.1 数据写入流程 (改造后)

用户在工厂A提交报工单:

1. Controller 接收请求

2. `siteGuard.middleware` 校验 → 自动注入 `site_prefix='A'`

3. 编号生成 → `WR-A-20260515-001`

4. `INSERT work_report (... , site_prefix='A', version=1)`

5. `INSERT sync_log (table='work_report', record_id='WR-A-20260515-001', operation='C', source_site='A', status='pending')`

6. `EventBus.emit('WorkReportCreated', {... , site_prefix: 'A'})`

7. 同步服务检测到 `pending sync_log` → 推送到 `SYNC_PEERS`

8.2 数据同步流程

定时轮询 (每30秒):

1. 查询 `sync_log WHERE status='pending' AND retry_count < 3`

2. 按 `target_site` 分组打包

3. `POST /api/sync/push` → 各 peer

4. peer 接收后:

a. 检查 `version`: 若本地 `version` $\geq$ 远程 `version` → 冲突, 写入 `sync_conflict_log`

b. 若本地不存在或 `version` $<$ 远程 → `UPSERT + version = 远程 version`

c. 写入 `sync_log` 本地 (标记 `source_site` 为来源)

5. 推送成功 → 更新 `sync_log.status = 'synced'`

6. 推送失败 → `retry_count++`, 下次重试

8.3 各数据类型同步策略

数据类型	涉及表	归属	同步方向	同步频率
基础数据（物料、客户、供应商、BOM、工艺路线等）	item_master, customer, supplier, bom_header, routing_header 等	任一站点可创建	全 Mesh 广播	实时（变更即推）
生产数据（计划、报工、入库）	Production_plan, work_report, production_inbound_order 等	创建站点归属	全 Mesh 广播	准实时（30秒）
库存数据	material_inventory, lineside_inventory 等	仓库所属站点	全 Mesh 广播	准实时（30秒）
销售订单	sales_order, sales_order_detail	创建站点归属	全 Mesh 广播	实时
采购订单	purchase_order, purchase_req	创建站点归属	全 Mesh 广播	准实时
质量数据	production_inspection, nonconforming_product 等	检验站点归属	全 Mesh 广播	准实时
用户/权限	users, role, permission 等	任一站点可改	全 Mesh 广播	实时
财务数据	standard_cost_header, piece_rate_wage_header 等	创建站点归属	全 Mesh 广播	每小时
文件附件	uploads/ 目录	上传站点归属	rsync/API	每5分钟

8.4 冲突处理策略

冲突场景	发生概率	处理方式
两站同时修改同一基础数据（如同名客户）	中	乐观锁：version 不匹配 → 拒绝后到达的变更，写入 sync_conflict_log，提示用户手动合并
网络分区后，同一记录被两地修改	低	LWW (Last-Write-Wins)：以 updated_at 时间戳为准，自动合并，记录到 sync_conflict_log 供人工复核
编号重复	极低	站点前缀天然隔离，不冲突
从节点上线后大量积压同步	低	全量快照 + 增量：新节点先 GET /api/sync/snapshot 获取全量，然后切换到增量同步

8.5 汇总报表查询方式

```
-- 总部用户查看全系统生产计划（查本地库，零跨库延迟）
SELECT 'HQ' AS site, * FROM Production_plan WHERE site_prefix = 'HQ'
UNION ALL
SELECT 'A' AS site, * FROM Production_plan WHERE site_prefix = 'A'
UNION ALL
SELECT 'B' AS site, * FROM Production_plan WHERE site_prefix = 'B'
UNION ALL
SELECT 'C' AS site, * FROM Production_plan WHERE site_prefix = 'C'
ORDER BY production_number;

-- 工厂A用户执行同一条 SQL，同样能看到全系统数据
-- （因为工厂A也同步了 HQ/B/C 的数据）
→ 每个节点都有完整副本
```

九、实施优先级与分期建议

Phase 1: 基础准备 (2-3 周)

序号	任务	涉及文件	工作量
1.1	新增 <code>site.config.ts</code> , <code>sync.config.ts</code>	新增2文件	0.5天
1.2	<code>.env</code> 增加 <code>SITE_PREFIX</code> , <code>SYNC_PEERS</code> 等变量	<code>.env</code>	0.5天
1.3	编号生成统一改造 (18个函数 + 27处内联迁移)	<code>documentNumber.service.ts</code> + 27个 controller	5天
1.4	所有业务表增加 <code>site_prefix</code> , <code>version</code> , <code>last_synced_at</code>	<code>models/index.ts</code> 迁移脚本	3天
1.5	新增 <code>siteGuard.middleware.ts</code>	新增1文件	1天
1.6	健康检查端点扩展	<code>app.ts</code>	0.5天

Phase 2: 同步模块开发 (3-4 周)

序号	任务	涉及文件	工作量
2.1	<code>sync_log</code> / <code>sync_conflict_log</code> / <code>sync_peer_status</code> / <code>sync_checkpoint</code> 建表	迁移脚本	1天
2.2	同步服务核心 (变更捕获 + push/pull)	<code>modules/sync/*</code> 新增4-5文件	5天
2.3	冲突检测与自动合并 (LWW + 乐观锁)	<code>modules/conflict/*</code> 新增2-3文件	3天
2.4	EventBus 桥接同步: 业务事件自动写入 <code>sync_log</code>	<code>eventBus.ts</code> 改造	2天
2.5	定时任务分布式锁	<code>stockCountScheduler.service.ts</code>	1天
2.6	EventWorker 分布式锁	<code>eventWorker.service.ts</code>	1天

Phase 3: 前端适配 (2 周)

序号	任务	涉及文件	工作量
3.1	API 地址动态化	<code>request.ts</code> (PC + Mobile)	1天
3.2	站点标识展示 + 数据过滤	顶部导航组件 + 列表页	3天
3.3	非归属数据只读保护	各列表/编辑页	2天
3.4	同步状态监控页面	新增页面	2天

Phase 4: 集成测试与部署 (2-3 周)

序号	任务	工作量
4.1	双节点联调测试 (HQ + Factory A)	3天
4.2	断网恢复测试	2天
4.3	冲突场景测试	2天
4.4	性能压测 (同步延迟、数据库负载)	2天
4.5	部署自动化脚本	3天

总预估工期: 9-12 周

十、关键风险与应对

风险	影响	应对措施
40+ 表加 site_prefix 字段, 历史数据需回填	大表 ALTER 可能锁表	分批执行, 低峰期操作; 默认值填充当前站点前缀
编号格式变更, 前端/报表中硬编码的编号解析失效	现有数据编号无站点前缀	新旧编号兼容: 旧数据 site_prefix 回填但编号不变, 仅新数据使用新格式
全量数据同步导致网络/磁盘压力	每节点存储 = 单厂数据 × 节点数	橡胶密封件行业数据量小, 4节点约200GB, 普通服务器可承载
同步延迟导致用户看到过期数据	操作失误	前端展示数据来源站点 + 最后同步时间; 非归属数据加"同步于XX:XX"标签
同步服务单点故障	某个节点同步中断	sync_log 持久化 + 重试机制; 节点恢复后自动补偿积压数据
网络分区期间多节点同时修改基础数据	数据冲突	乐观锁 + LWW 自动合并 + 人工仲裁兜底

十一、文件改造清单总览

文件路径	改造内容	优先级
server/.env	新增 SITE_PREFIX, DEPLOY_MODE, SYNC_PEERS	P0
server/src/config/site.config.ts	新增：站点配置管理	P0
server/src/config/sync.config.ts	新增：同步配置，含 SYNC_PEERS 对等节点列表	P1
server/src/config/database.ts	连接池放大 + 超时配置	P0
server/src/app.ts	健康检查扩展 + 同步路由 + 站点守卫	P0
server/src/server.ts	实例标识、启动时注册到 Mesh	P0
server/src/middleware/siteGuard.middleware.ts	新增：站点写入守卫	P0
server/src/services/documentNumber.service.ts	18个函数站点前缀化 + 统一基础函数	P0
27个 controller 文件	内联编号生成迁移到 documentNumber.service.ts + 站点前缀	P0
server/src/models/index.ts	40+ 业务表增加 site_prefix / version / last_synced_at 迁移	P0
server/src/shared/eventBus.ts	事件发布时写入 sync_log + 新增同步事件	P1
server/src/services/eventStore.service.ts	增加 source_site 字段 + 分布式锁	P1
server/src/services/eventWorker.service.ts	分布式锁 + site_prefix 过滤	P1
server/src/services/stockCountScheduler.service.ts	分布式锁防重	P1
server/src/middleware/upload.middleware.ts	文件元数据记录 + 同步策略	P1
server/src/modules/sync/*	新增：数据同步模块 (push/pull/snapshot/peer)	P1
server/src/modules/conflict/*	新增：冲突处理模块	P1
server/src/modules/report-aggregation/*	新增：汇总报表模块	P1
client/src/utlis/request.ts	API 地址动态化 + 站点请求头	P0
client-mobile/src/api/request.ts	API 地址动态化 + 站点请求头	P0
前端各列表/编辑页	站点标识展示 + 非归属数据只读保护	P1
ecosystem.config.js	多实例 cluster 模式	P0